

Adding Transport Times to the FJSP with Batching: Formulation and Graph Representations

Mariana Correia

September 27, 2026

This document extends `docs/batching_formulation.tex`. It uses the same notation and refers to its equations as (F1)–(F2), (B1)–(B7'). Section 10 collects the critical assessment of the design and the open decisions.

Contents

1	Transport constraint	1
1.1	Shop-floor layout	1
1.2	Assumptions	1
2	Mixed-integer formulation	2
3	Consequences for the decision process	3
3.1	State and dynamics	3
3.2	Batching features under transport	3
3.3	Action-edge features	4
4	Transport information to represent	4
5	Transport representation T-0: none	5
6	Transport representation T-F: features	5
7	Transport representation T-E: auxiliary edges	6
8	Experimental grid	7
9	Metrics that must change	8
10	Critical assessment and open decisions	8
10.1	Weaknesses of the proposal	8
10.2	Decisions to confirm	9
11	Calibration of the transport magnitude	10
12	Implementation	10
13	Illustrative example	11

1 Transport constraint

1.1 Shop-floor layout

Every machine $\mu \in \mathcal{M}$ has a position $(X_\mu, Y_\mu) \in \mathbb{Z}_{\geq 0}^2$ on the shop floor. Coordinates are measured in *time units*, the same unit as the processing times $p_{o,\mu}$. There is also one load/unload station (depot), denoted $\mu = 0$, with position (X_0, Y_0) . Let $\mathcal{M}_0 = \mathcal{M} \cup \{0\}$. The transport time between two locations is their Manhattan distance:

$$\tau_{\mu\nu} = |X_\mu - X_\nu| + |Y_\mu - Y_\nu|, \quad \mu, \nu \in \mathcal{M}_0. \quad (1)$$

τ is a metric: $\tau_{\mu\mu} = 0$, $\tau_{\mu\nu} = \tau_{\nu\mu}$ and $\tau_{\mu\nu} \leq \tau_{\mu\kappa} + \tau_{\kappa\nu}$. Because of the triangle inequality, routing a job through an intermediate location never shortens its transport. Manhattan (rather than Euclidean) distance models material that moves along perpendicular aisles. With integer coordinates every $\tau_{\mu\nu}$ is an integer, which keeps the CP-SAT model exact.

1.2 Assumptions

- (T1) **Unlimited transport capacity.** There are no vehicles (AGVs, robots or operators) to schedule. Any number of jobs can be transported at the same time. Transport is therefore a *machine-dependent time lag* between consecutive operations of a job.
- (T2) Transport starts as soon as the operation (or its batch) finishes. Loading and unloading times are zero or are included in τ .
- (T3) Buffers are unlimited. A machine becomes free when its batch finishes, because the transport does not occupy it. A job that arrives before its machine is free waits in the buffer.
- (T4) If two consecutive operations of a job run on the same machine, no transport is needed ($\tau_{\mu\mu} = 0$).
- (T5) Every job starts at the depot. The transport from the depot to the machine of its first operation is counted. The return of the finished job to the depot is *not* counted in the makespan (see Section 10, decision D2).
- (T6) Batch members are transported *individually*. After a batch finishes on μ , all of its members are located at μ , and each one travels to the machine of its own next operation.

For an operation o let $\mu(o)$ be the machine that processes it. The transport constraint replaces the routing precedence (B6) by

$$S_{O_{j,k+1}} \geq C_{O_{j,k}} + \tau_{\mu(O_{j,k}), \mu(O_{j,k+1})}, \quad S_{O_{j,1}} \geq \tau_{0, \mu(O_{j,1})}. \quad (2)$$

Consequently, a batch b on machine μ cannot start before its last member has arrived:

$$S_b \geq \max_{o \in b} \left(C_{\text{pred}(o)} + \tau_{\mu(\text{pred}(o)), \mu} \right), \quad (3)$$

where $\text{pred}(o)$ is the previous operation of job(o), with $C = 0$ and location 0 for a first operation.

2 Mixed-integer formulation

The batching model is kept unchanged. Only the machine of each operation and the transport lags are added.

Additional variables. $w_{o,\mu} \in \{0, 1\}$: operation o is processed on machine $\mu \in \mathcal{M}_o$.

Additional constraints.

$$w_{o,\mu} \geq x_{o,b} + y_{b,\mu} - 1 \quad \forall o \in \mathcal{O}, b \in \mathcal{B}_{\varphi(o)}, \mu \in \mathcal{M}_{\varphi(o)} \quad (\text{T-a})$$

$$\sum_{\mu \in \mathcal{M}_o} w_{o,\mu} = 1, \quad w_{o,\mu} = 0 \quad \forall \mu \notin \mathcal{M}_o \quad \forall o \in \mathcal{O} \quad (\text{T-b})$$

$$S_{O_{j,k+1}} \geq C_{O_{j,k}} + \tau_{\mu\nu}(w_{O_{j,k},\mu} + w_{O_{j,k+1},\nu} - 1) \quad \forall j, k < n_j, \mu \neq \nu \quad (\text{T-c})$$

$$S_{O_{j,1}} \geq \sum_{\nu \in \mathcal{M}_{O_{j,1}}} \tau_{0\nu} w_{O_{j,1},\nu} \quad \forall j \in \mathcal{J} \quad (\text{T-d})$$

In (T-c), $\mu \in \mathcal{M}_{O_{j,k}}$ and $\nu \in \mathcal{M}_{O_{j,k+1}}$. The big constant becomes $L = \sum_o \max_{\mu} p_{o,\mu} + \sum_j n_j \max_{\mu,\nu \in \mathcal{M}_0} \tau_{\mu\nu}$.

Interpretation.

- (T-a)–(T-b): by (B1) and (B4), exactly one batch b contains o and exactly one machine μ^* has $y_{b,\mu^*} = 1$. (T-a) forces $w_{o,\mu^*} = 1$ and (T-b) forces all other $w_{o,\mu}$ to 0. So w is exactly the machine of o , without a bilinear term.
- (T-c): if $O_{j,k}$ runs on μ and $O_{j,k+1}$ on ν , the bracket equals 1 and the lag $\tau_{\mu\nu}$ is enforced. Otherwise the bracket is ≤ 0 and the constraint is weaker than (B6), which is kept. No big- M is needed. The number of constraints is $\sum_j \sum_{k < n_j} |\mathcal{M}_{O_{j,k}}| |\mathcal{M}_{O_{j,k+1}}| \leq |\mathcal{O}| m^2$.
- (T-d): transport from the depot to the first machine.
- (3) does not need its own constraint. It follows from (B5a), which sets $S_o = S_b$ for every member, together with (T-c).

With $\tau \equiv 0$ (all machines and the depot at one point) the model reduces exactly to the batching model. This is also the main regression test for the implementation (Section 12).

CP-SAT version. In `src/batch_solver.py` each potential batch already has one presence literal per eligible machine. For every operation o and machine μ , a literal $z_{o,\mu}$ is created with $x_{o,b} \wedge y_{b,\mu} \Rightarrow z_{o,\mu}$ and $\sum_{\mu} z_{o,\mu} = 1$. Then (T-c) becomes

```
model.add(S[c] >= C[a] + tau[mu][nu])
        .only_enforce_if([z[a,mu], z[c,nu]])
```

The horizon must include the transport term of L .

3 Consequences for the decision process

3.1 State and dynamics

The environment keeps, for every active job j , its completion time C_j (the end of its last scheduled operation, 0 at the start) and its *location* $\ell_j \in \mathcal{M}_0$ ($\ell_j = 0$ at the start). The arrival time of job j at machine μ and its earliest start there are

$$a_{j\mu} = C_j + \tau_{\ell_j\mu}, \quad s_{j\mu} = \max(F_{\mu}, a_{j\mu}), \quad (4)$$

where F_{μ} is the free time of μ . Without transport, $a_{j\mu} = C_j$ for every μ . With transport, **readiness depends on the machine**. This one change affects most of the batching machinery.

Batch completion rule. The selected pair (μ, j) opens a batch starting at $S_{bt} = s_{j\mu}$. A candidate j' now joins only if it has *arrived at* μ by then:

$$\varphi(o_{j'}^t) = f, \quad \mu \in \mathcal{M}_{o_{j'}^t}, \quad a_{j'\mu} \leq S_{bt}, \quad (5)$$

together with the capacity and order-difference conditions, adding candidates in order of arrival time $a_{j'\mu}$ (previously: ready time). The property “joining never delays the batch” is kept.

Update after a decision. For every member j of the batch: $C_j \leftarrow S_{b_t} + p_{b_t}$ and $\ell_j \leftarrow \mu$. For every other job, $s_{j\mu}$ is updated through F_μ as before.

Reward, mask and teacher. The reward (makespan increase) is unchanged. The mask and the warm-start teacher keep their definitions (earliest start, tie-break earliest completion). Because they are computed from $s_{j\mu}$ in (4), they are *transport-aware* in all representations, including the one without transport information.

3.2 Batching features under transport

The batching features of the batching document assume a machine-independent ready time r_j . They must be re-evaluated at a specific machine. Let

$$\mu_j^* = \arg \min_{\mu \in \mathcal{M}_{o_j^t}} s_{j\mu} \quad (\text{ties: smallest } s_{j\mu} + p_{o_j^t, \mu}), \quad (6)$$

be the machine at which job j can start earliest. Then

$$\pi_j(t) = \min \left(B_{\varphi(o_j^t)} - 1, \left| \{j' \neq j : \varphi(o_{j'}^t) = \varphi(o_j^t), |\kappa(o_{j'}^t) - \kappa(o_j^t)| \leq \Delta, \right. \right. \\ \left. \left. \mu_j^* \in \mathcal{M}_{o_{j'}^t}, a_{j'\mu_j^*} \leq s_{j\mu_j^*} \} \right| \right). \quad (7)$$

This matches the batch completion rule at μ_j^* exactly. The old condition $\mathcal{M}_{o_j^t} \cap \mathcal{M}_{o_{j'}^t} \neq \emptyset$ is replaced by the stricter $\mu_j^* \in \mathcal{M}_{o_{j'}^t}$. For the family node (Representation A), t_f is the smallest $s_{j\mu_j^*}$ over the current members, μ_f the machine that achieves it, and $R_f(t) = \{o_{j'}^t \in \mathcal{O}^f : \mu_f \in \mathcal{M}_{o_{j'}^t}, a_{j'\mu_f} \leq t_f\}$.

Without transport ($\tau \equiv 0$) readiness does not depend on the machine, and the original definitions of the batching document are kept unchanged. This keeps the batching-only environments, and the models already trained on them, exactly as they were.

Critical note. These features are now *approximations*: they describe the batch at one machine (μ_j^* or μ_f), while the policy may choose another. Exact per-machine values belong on per-machine edges. In Representation A the `batch_exec` edge (f, μ) has three unused (zero) columns, which could hold $n_{f\mu}(t)/B_f$ and the earliest batch start at μ . That, however, changes the batching representation between the experiments with and without transport. For this reason it is recommended only as a later refinement.

3.3 Action-edge features

The action edge (μ, j) keeps its five columns $[p_b, \text{rel. speed}, s_{j\mu}, s_{j\mu} + p_b, s_{j\mu} - F_\mu]$, now with the transport-aware $s_{j\mu}$ of (4) and the batch rule above. A sixth column is added (Section 6):

$$\mathbf{a}_{\mu j} = \left[p_b, \frac{p_{o_j^t, \mu}}{\sum_{\mu'} p_{o_j^t, \mu'}}, s_{j\mu}, s_{j\mu} + p_b, s_{j\mu} - F_\mu, \theta \tau_{\ell_{j\mu}} \right] \in \mathbb{R}^6, \quad (8)$$

with $\theta = 1$ in the feature representation and $\theta = 0$ otherwise. Every attributed relation of the transport environments therefore has dimension 6: the other relations are padded with zero columns. The edge width is a property of the environment (`EDGE_DIM`), and the GNN reads it when it is built. The batching-only environments keep width 5. Column 5, $s_{j\mu} - F_\mu$, now also counts machine idle time *caused by transport*, i.e. the job has not yet arrived.

4 Transport information to represent

For the comparison to be fair, both representations carry the *same* information and differ only in how it is encoded, as Representations A and B did for batching. Four pieces of information are identified:

I1 – Layout.

Positions of the machines and distances between them (static).

I2 – Location.

Where each active job is now, ℓ_j (dynamic).

I3 – Immediate transport.

The transport each candidate action would cause, $\tau_{\ell_j\mu}$ for $\mu \in \mathcal{M}_{o_j^t}$ (dynamic, job-machine pair).

I4 – Future transport.

The transport that the pending operations will require, given their eligible machines (static for pending operations).

For I4, define for two consecutive operations $o = O_{j,k}$ and $o^+ = O_{j,k+1}$

$$\tau_{o \rightarrow o^+}^{\min} = \min_{\mu \in \mathcal{M}_o, \nu \in \mathcal{M}_{o^+}} \tau_{\mu\nu}, \quad \tau_{o \rightarrow o^+}^{\text{avg}} = \frac{1}{|\mathcal{M}_o| |\mathcal{M}_{o^+}|} \sum_{\mu \in \mathcal{M}_o} \sum_{\nu \in \mathcal{M}_{o^+}} \tau_{\mu\nu}. \quad (9)$$

$\tau^{\min}$ is the unavoidable transport (if all machines have distinct positions, it is positive exactly when the two operations share no eligible machine). τ^{avg} is the transport expected under a machine choice that ignores the layout. For the first operation, $\mathcal{M}_o$ is replaced by $\{0\}$ (the depot).

Table 1: Where each piece of information lives in each transport representation.

	T-F: features	T-E: auxiliary edges	T-0
I1 layout	machine $(X_\mu, Y_\mu) + \tau$ on listens edges	transport relation machine→machine	–
I2 location	job (X_{ℓ_j}, Y_{ℓ_j})	at relation job→machine	–
I3 immediate	6th column of the action edge	reach relation job→machine	implicit in $s_{j\mu}$
I4 future	operation features τ^{in}	transfer relation op→op	–

5 Transport representation T-0: none

The graph is the one of the chosen batching representation. There is no position, no location and no distance, and the sixth action-edge column is 0. The environment, the mask and the teacher still apply the transport constraint (2).

Critical note. T-0 is *not* blind to transport. The earliest start $s_{j\mu}$ on the action edges already includes $\tau_{\ell_j\mu}$, because the start time is a property of the action rather than of the representation. Hiding it (showing $\max(F_\mu, C_j)$ instead) would make the edges report start times that are wrong, and the teacher, which ranks by the true $s_{j\mu}$, could no longer be imitated. That is the problem already fixed for batching. The consequence is that T-F and T-E are measured against a baseline that already sees the *current* effect of transport. What they add is I1, I2, I4 and the *isolated* value of I3 (the start time mixes transport with machine availability). The expected gain is therefore smaller than for batching, and the hypothesis should be stated as: “explicit transport information improves decisions whose consequences appear in *later* transports”.

6 Transport representation T-F: features

No node type and no relation is added. Transport information is stored as features of the existing nodes and edges.

Machine node.

$$\mathbf{x}_\mu^T = [\mathbf{x}_\mu \parallel \tilde{X}_\mu \parallel \tilde{Y}_\mu]. \quad (10)$$

Job node. The position of the machine where the job currently is (the machine of its last processed operation, or the depot):

$$\mathbf{x}_j^T(t) = [\mathbf{x}_j(t) \parallel \tilde{X}_{\ell_j} \parallel \tilde{Y}_{\ell_j}]. \quad (11)$$

Operation node. The transport *into* the operation:

$$\mathbf{x}_o^T(t) = [\mathbf{x}_o \parallel \tau_o^{\text{in},\min}(t) \parallel \tau_o^{\text{in},\text{avg}}(t)], \quad (12)$$

where

$$\tau_o^{\text{in},\cdot}(t) = \begin{cases} \tau_{\text{pred}(o) \rightarrow o} & \text{if } \text{pred}(o) \text{ is not scheduled,} \\ \min / \text{avg}_{\nu \in \mathcal{M}_o} \tau_{\ell_{\text{job}(o)}\nu} & \text{if } o \text{ is the current operation of its job,} \end{cases} \quad (13)$$

with τ from (9). The feature is static for pending operations and becomes exact (known origin) when the operation becomes current.

Critical note. The original idea was an operation feature “describing what was processed”. It would carry no information: scheduled operations are *removed* from the graph (`_remove_operations`), so the network never sees a processed operation. What was processed, and where, is kept by the job node through ℓ_j . The operation feature is therefore redefined to look ahead, which is where the operation node is useful.

Machine-machine edges. The existing fully connected relation (machine, `listens`, machine) gets an attribute:

$$\mathbf{a}_{\mu\nu} = [\tau_{\mu\nu}, 0, 0, 0, 0, 0]. \quad (14)$$

Action edges. The sixth column of (8) holds $\tau_{\ell_j\mu}$ ($\theta = 1$).

Normalisation of the coordinates. $\tilde{X} = (X - X_{\min})/W$ and $\tilde{Y} = (Y - Y_{\min})/W$, with $W = \max(X_{\max} - X_{\min}, Y_{\max} - Y_{\min})$ computed over $\mathcal{M}_0$. The scale is the same for the two axes and for jobs and machines. These columns must be *excluded* from the per-column min–max normalisation of `normalize_state`.

Critical note. `normalize_state` rescales every column to $[-1, 1]$ separately for each node type. The job x -coordinate would be scaled by the extent of the *job locations* and the machine x -coordinate by the extent of the *machines*, so the same physical point would get two different values, and the network could not relate a job to a machine by position. The shared scale W avoids this, and one scale for both axes keeps the Manhattan geometry.

Critical note. Two limitations of the features should be discussed in the dissertation. (i) Absolute coordinates are not translation invariant. Only differences matter, so the coordinates are mainly useful when combined, e.g. in the actor, which concatenates job and machine embeddings. This is why I3 is also given explicitly. (ii) The distance on `listens` is a deterministic function of the two endpoint coordinates and says nothing about any job. It helps a machine weigh its neighbours, e.g. where its jobs will go next, but the quantity the decision actually needs, $\tau_{\ell_j\mu}$, is a job–machine value. Strictly speaking, T-F is “features on existing nodes *and* existing edges”.

7 Transport representation T-E: auxiliary edges

No node feature is added. Transport information is encoded by new relations whose attributes are transport times. The same information is used as in T-F.

$$\mathcal{E}_{MM}^{\text{tr}} = \{(\mu, \nu) : \mu \neq \nu, \exists j, k : \mu \in \mathcal{M}_{O_{j,k}}, \nu \in \mathcal{M}_{O_{j,k+1}}\}, \quad (15)$$

$$\mathcal{E}_{JM}^{\text{at}}(t) = \{(j, \ell_j) : j \text{ active}, \ell_j \neq 0\}, \quad (16)$$

$$\mathcal{E}_{JM}^{\text{reach}}(t) = \{(j, \mu) : j \text{ active}, \mu \in \mathcal{M}_{o_j^t}\}, \quad (17)$$

$$\mathcal{E}_{OO}^{\text{tf}}(t) = \{(o^+, o) : o = O_{j,k}, o^+ = O_{j,k+1}, o \text{ not scheduled}\}. \quad (18)$$

Table 2: New relations of T-E and their attributes (padded with zeros to dimension 6).

Edge set	Relation	Attribute	Info
(15)	(machine, transport , machine)	$[\tau_{\mu\nu}]$	I1
(16)	(job, at , machine) + reverse hosts	none	I2
(17)	(job, reach , machine)	$[\tau_{\ell_j\mu}]$	I3
(18)	(operation, transfer , operation)	$[\tau_{o \rightarrow o^+}^{\min}, \tau_{o \rightarrow o^+}^{\text{avg}}]$	I4

The action edge keeps $\theta = 0$.

Interpretation.

- **transport** (I1) is a *sparse* layout graph. Only machine pairs that some routing can actually use are connected, unlike the fully connected **listens**. Both directions are included (τ is symmetric).
- **at/hosts** (I2) gives the location as structure. A machine learns which jobs are waiting there, which matters for batching, because members of the same batch leave from the same machine (assumption (T6)). A job at the depot has no **at** edge. Its origin is still visible through the **reach** attributes.
- **reach** (I3) connects the same pairs as the action edges but is a separate relation with its own attention. The actor reads edge attributes *only* from the action edges, so in T-E the value $\tau_{\ell_j\mu}$ reaches the decision only through the embeddings. This is the real difference from T-F for I3: T-F gives it to the scorer directly, T-E makes the GNN use it.
- **transfer** (I4) runs parallel to **prec** and in the same direction (successor $\rightarrow$ predecessor). The current operation therefore receives the transport of the downstream part of its routing, like the transport-weighted conjunctive arcs of a disjunctive graph.

Size. $|\mathcal{E}_{MM}^{\text{tr}}| \leq m(m-1)$, $|\mathcal{E}_{JM}^{\text{at}}| \leq n$, $|\mathcal{E}_{JM}^{\text{reach}}|$ equals the number of action edges ($\leq nm$), and $|\mathcal{E}_{OO}^{\text{tf}}| \leq |\mathcal{O}| - n$. The graph grows linearly in the instance size.

Receptive field. With the two GAT layers used, every piece of information reaches the scored nodes: I2 and I3 in one hop into the job and machine of the action, I4 in one hop into the current operation and a second hop into its job, and I1 in one hop between machines.

Critical note. The heterogeneous model aggregates relations with a *mean* (`to_hetero(..., aggr='mean')` in `src/bopo.py`). Adding **at**, **reach** and **hosts** changes the number of relations arriving at job and machine nodes, and therefore the weight of every existing message. Part of any difference between T-E and T-F may come from this dilution rather than from the transport information. The same confound exists between batching Representations A and B. It should be

named as a threat to validity, or controlled by one ablation that adds the T-E relations with all attributes set to zero.

Critical note. `at` and `reach` partly overlap: given the edge `at` and the `transport` attributes, $\tau_{\ell_j\mu}$ is determined. They are kept separate because a 2-layer GNN cannot reliably reconstruct a pairwise distance from averaged embeddings. If edges have to be saved, `at` is the first candidate to remove.

8 Experimental grid

Batching and transport are represented independently, which gives nine configurations:

	T-0	T-F	T-E
batching baseline (<code>ojmb_base</code>)	<code>base+t0</code>	<code>base+tf</code>	<code>base+te</code>
Representation A (<code>ojmb_node</code>)	<code>node+t0</code>	<code>node+tf</code>	<code>node+te</code>
Representation B (<code>ojmb_edge</code>)	<code>edge+t0</code>	<code>edge+tf</code>	<code>edge+te</code>

All nine configurations solve *the same problem* (batching + transport) on the *same instances*, with the same action space, batch rule, mask, teacher and reward. They differ only in the graph. `base+t0` is the full baseline.

- **Main effects.** The transport effect is read along the rows and the batching effect along the columns. Because all configurations are evaluated on the same instances, the comparisons are *paired*. Friedman’s test over the nine configurations, followed by pairwise Wilcoxon signed-rank tests with a multiple-comparison correction, is recommended.
- **Interaction.** If the best transport representation depends on the batching representation, this is itself a result (e.g. `hosts` edges may help most together with the family node).
- **Seeds.** At least three training seeds per configuration. With nine configurations this is 27 training runs, so the per-run budget has to be fixed before the grid is started.

Instance generation and calibration. Coordinates are drawn uniformly from $\{0, \dots, L\}^2$, with the depot drawn the same way. For points uniform on $[0, L]^2$, $\mathbb{E}|X_\mu - X_\nu| = L/3$, so the expected transport time is $\mathbb{E}[\tau] = 2L/3$. To make transport a fraction ρ of the mean processing time $\bar{p}$,

$$L = \left\lceil \frac{3}{2} \rho \bar{p} \right\rceil, \quad \text{e.g. } \bar{p} \approx 50, \rho = 0.3 \Rightarrow L = 23. \quad (19)$$

Critical note. If ρ is too small, transport is irrelevant and every configuration will perform the same, which cannot be told apart from “the representation does not help”. If ρ is too large, transport dominates and the batching signal is lost. Before training, the CP-SAT solutions for a few values of ρ (e.g. 0.1, 0.3, 0.5) should be used to report (i) the relative increase of the makespan over the transport-free instance and (ii) the fraction of job time spent in transport. The value of ρ is then chosen so that transport is clearly present but not dominant. The same statistics also go in the dissertation, to show that the problem actually changed.

9 Metrics that must change

Feasibility check. Add the transport constraint to `check_schedule` (batching solver and `src/metrics`): $S_c \geq C_a + \tau_{\mu(a)\mu(c)}$ for consecutive a, c , and $S_{O_{j,1}} \geq \tau_{0\mu(O_{j,1})}$.

Lower bound. The machine-set bound (b) of `lower_bound` is still valid, because transport does not occupy machines. The job bound (a) must include transport, otherwise it is loose and the relative error to the lower bound is inflated. The tightest cheap bound is a shortest path through the layered graph of eligible machines:

$$\text{LB}_j = \min_{\nu_1, \dots, \nu_{n_j}} \left(\tau_{0\nu_1} + \sum_{k=1}^{n_j} p_{O_{j,k}, \nu_k} + \sum_{k=1}^{n_j-1} \tau_{\nu_k \nu_{k+1}} \right), \quad \nu_k \in \mathcal{M}_{O_{j,k}}, \quad (20)$$

computed by dynamic programming in $\mathcal{O}(n_j m^2)$. It is valid with batching, because a batch lasts at least as long as each of its members. Note that $\sum_k \min_{\nu} p_{O_{j,k}, \nu} + \sum_k \tau_{O_{j,k} \rightarrow O_{j,k+1}}^{\min}$ is also valid but weaker, since it may use different machines for the same operation in two terms.

References. All CP-SAT reference makespans must be recomputed with transport. The references of the batching experiments do not apply.

10 Critical assessment and open decisions

10.1 Weaknesses of the proposal

1. **Transport without resources.** Under (T1) the problem is an FJSP with batching and *machine-dependent time lags*, not an FJSP with transport resources (AGVs, robots), which is the more common meaning of “transport” in the literature. This is a legitimate and cleaner variant, but it must be named and justified as such. The justification is that it isolates the spatial effect of the layout, which is what the representations encode, without adding a new resource type. Adding vehicles would require new decisions (which vehicle, empty trips) and would change the action space. That is outside the scope of a representation study.
2. **Operation feature “what was processed”.** Processed operations are deleted from the graph, so this feature is never observed. It is replaced by the incoming transport (Section 6).
3. **Machine identity as a job feature.** A machine index is a label that only has meaning inside one instance, the same argument used to reject the family index. The location is given as coordinates (T-F) or as an edge (T-E).
4. **Coordinate normalisation.** Without the shared scale W , job and machine coordinates live in different frames (Section 6). This is the error most likely to silently void T-F.
5. **Machine-machine distance.** It is static, and so far removed from the decision that on its own it is unlikely to matter. It is kept in both representations (I1) for completeness, but the explicit job-machine transport (I3) is the main component. An ablation without I1 would show whether it contributes anything.
6. **The baseline sees transport.** See Section 5. Expect smaller effects than for batching and phrase the hypothesis accordingly.
7. **Batching becomes machine-dependent.** The completion rule, π_j and the family-node features change definition, and the batching features become approximations evaluated at one machine. The batch rule also stays myopic. It may add a partner to a batch on a machine that is far from that partner’s next machine, increasing its future transport. The policy has no control over this. It is a property of the environment and must be stated.

8. **Edge dimension.** The transport environments use width 6 instead of 5, so models trained for batching only cannot be reused on them. This is acceptable because the problem is different anyway, but the batching-only results and the transport results are not directly comparable.
9. **Aggregation confound** in T-E (Section 7).
10. **Calibration.** Without the ρ study, a null result cannot be interpreted (Section 8).
11. **Cost.** Nine configurations with several seeds. If the budget is short, the most informative subset is the batching representation that won the batching experiments, combined with T-0, T-F and T-E.

10.2 Decisions to confirm

D1 – Depot.

Recommended: include a depot and the first transport. It gives every job a defined location at $t = 0$, which the job feature needs. Alternative: no depot and $\tau = 0$ for the first operation. This is simpler, but then the location of an unstarted job is undefined and needs a flag.

D2 – Return to the depot.

Recommended: not counted. Counting it adds $\tau_{\mu(O_{j,n_j}),0}$ to the completion of each job and makes the choice of the last machine matter.

D3 – Transport magnitude ρ .

Calibrated in Section 11: $\rho = 0.3$.

D4 – I3 in T-F.

Recommended: on the action edge (direct access by the scorer). Alternative: omit it, and let the actor derive it from the job and machine coordinates. This is a stricter “node features only” variant, but weaker.

D5 – Batch features per machine.

Recommended for now: evaluate at μ_j^* / μ_f and keep the batching representation unchanged. A per-machine version on the `batch_exec` edges is a later refinement.

11 Calibration of the transport magnitude

Eight batching instances of the training size (8–10 jobs, 5–10 machines, 5–6 operations per job) were solved by CP-SAT without transport and with the layouts of four values of ρ . The layout is drawn after the rest of the instance, so the jobs, families and processing times are the same for every ρ . All 40 solutions were proven optimal within 20 s. The *increase* is $C_{\max}(\rho)/C_{\max}(0) - 1$. The *share* is the fraction of the jobs’ time spent travelling in the optimal schedule, $\sum_j(\text{transport of } j)/\sum_j C_j$.

Table 3: Effect of ρ on the optimal makespan (8 instances, mean $C_{\max}(0) = 343.8$).

ρ	increase (mean)	increase (min–max)	share
0.1	0.055	0.029–0.102	0.061
0.3	0.140	0.081–0.226	0.145
0.5	0.198	0.118–0.325	0.170
1.0	0.425	0.293–0.606	0.300

$\rho = 0.1$ changes the makespan by about 5%, which is within the typical gap of a learned policy, so the representations could not be told apart. $\rho = 1$ makes transport the dominant cost (30% of the jobs' time) and would drown the batching signal. $\rho = 0.3$ adds 14% to the makespan, with transport 15% of the jobs' time. It is chosen as the transport scale. The share grows more slowly than ρ (0.17 at $\rho = 0.5$) because the optimal schedules increasingly keep consecutive operations on the same or nearby machines. This is exactly the behaviour the representations should help the policy learn. The calibration uses only eight instances and should be repeated on the final dataset (`python -m src.calibrate_transport`).

12 Implementation

Layout (`src/transport.py`).

Instance keys `"coords"` ($m \times 2$ integers), `"depot"` and `"transport_rho"`. It also provides the matrix τ (depot at index m), $\tau^{\min}$ and τ^{avg} of (9), and the job lower bound. Instances without a layout have $\tau \equiv 0$.

Instances (`src/batch_generator.py`).

Option `transport_rho`. Positions are uniform on $\{0, \dots, L\}^2$ with $L = \lceil 1.5 \rho \bar{p} \rceil$, drawn last. The dataset is built with `python -m src.generate_batching_dataset -transport-rho 0.3 -out data/transport`. Training instances use the ρ stored in the validation dataset.

Solver and checkers.

`src/batch_solver.py` implements (T-a)–(T-d) in CP-SAT, and its `check_schedule` reports transport and depot violations. `src/metrics/batching.py` adds the `transport` constraint to the batching checks. The job bound of `lower_bound` is the shortest path of Section 9, which reduces to the old bound without transport.

Environments (`src/env_batching.py`).

The transport dynamics (location ℓ_j , arrivals, transport-aware start times, batch rule with arrivals) are applied by every batching environment, so the three batching-only environments also solve transport instances correctly. A second class attribute `TRANSPORT_REP` selects the representation, and the nine combinations are registered as `ojmb_<node|edge|base>_<t0|tf|te>`. In T-F the coordinates are the last two columns of the machine and job features, and `normalize_state` keeps them on the shared scale, mapped to $[-1, 1]$. All attributed relations are padded to width 6 after normalisation.

Model (`src/bopo.py`, `src/gat.py`, `src/gine.py`, `src/transformer.py`).

The edge width of the GNN layers is taken from the environment (5 or 6).

Command line.

`-representation transport` trains the nine configurations. The test script only pairs a model with instances of its own kind (plain FJSP, batching, batching with transport).

Tests (`tests/test_transport.py`).

- (i) Regression: without a layout, and with every location at the same point, each of the nine environments takes exactly the same schedules as the batching-only environment for the same actions. T-0 also has identical features.
- (ii) Random and teacher rollouts of every environment pass both checkers, and the makespan never falls below the lower bound.
- (iii) On small instances the CP-SAT solution passes the checker and is never worse than the teacher, and the lower bound never exceeds the solver's bound.

- (iv) The hand-built instance of Section 13 gives the expected batches, arrival-based start times, T-F features and T-E edges.
- (v) Violations of the transport and depot constraints are detected.
- (vi) A model is built for all nine representations, evaluated with the metrics pipeline, and trained for one BOPO update.

The tests were checked against a mutation: when the environment ignores transport, (ii) and (iv) fail. The existing tests (`tests/test_metrics.py`) still pass. A two-step training run of `ojmb_node_tf`, `ojmb_edge_te` and `ojmb_base_t0` completed end to end.

13 Illustrative example

Three machines and a depot, with $M_1 = (0, 0)$, $M_2 = (4, 3)$, $M_3 = (10, 0)$ and depot $(0, 5)$:

	0	M_1	M_2	M_3
$\tau =$	0	5	6	15
M_1	5	0	7	10
M_2	6	7	0	9
M_3	15	10	9	0

Jobs 1 and 2 have just finished: job 1 on M_1 at $C_1 = 10$ and job 2 on M_3 at $C_2 = 12$. Their current operations belong to the same family f , with $B_f = 2$, satisfy the order difference, and are both eligible only on M_2 , which is free from $F_{M_2} = 15$. Arrivals: $a_{1,M_2} = 10 + 7 = 17$ and $a_{2,M_2} = 12 + 9 = 21$.

- Action $(M_2, 1)$: $S_b = \max(15, 17) = 17$. Job 2 arrives at $21 > 17$, so it does not join. Job 1 runs alone, and job 2 later needs its own batch.
- Action $(M_2, 2)$: $S_b = \max(15, 21) = 21$. Job 1 has arrived ($17 \leq 21$) and joins. One batch, but M_2 is idle from 15 to 21 because of transport.
- Without transport, both jobs would be ready at 15 and either action would form the batch. Transport makes the choice of *which* job opens the batch decisive. The batch-aware processing time p_b on the action edge already shows the difference (a single job vs. the longest of the pair), and so do the start times (17 vs. 21), even in T-0.
- After the batch, both jobs are at M_2 . If their next operations are on M_3 , both pay $\tau_{M_2M_3} = 9$. T-F shows this through the incoming transport of those operations, and T-E through the **transfer** edges. T-0 shows it only once the jobs are dispatched.

Waiting to batch is not always worth it. The test instance is a variant of this example: job 0 ends on M_1 at 10 and arrives at M_2 at 17, job 1 ends on M_3 at 16 and arrives at 25, and the family operations take 4 and 3. Batching them means starting at 25 and ending at 29. Running job 0 alone (17–21) and job 1 alone (25–28) ends at 28, which equals the lower bound of job 1 and is the CP-SAT optimum. Transport can therefore make the batch that the completion rule would form (when job 1 opens it) the *worse* choice. The policy has to learn when to accept that.